

Run a Real Web Server on Your Mac

With one Terminal command, your Mac becomes a web server. Your browser sends it real HTTP requests, and it sends back real responses with status codes. It's the same conversation your site has once it's published, so you can test everything on your own computer first.

Why run a server at all?

Double-clicking `index.html` opens it straight from your disk. The address bar then starts with `file:///`, and no server is involved. The browser simply reads the file, much like a text editor would. That works for a single page of plain HTML, but it isn't how the web works. On the web, every page, stylesheet and image arrives because the browser **asked a server** for it and the server **answered**.

A **local server** is a small web server program running on your own Mac. Your browser talks to it over HTTP, exactly as it talks to GitHub Pages or any other website, except the request never leaves your computer. That lets you:

- **See the real conversation.** Every request shows up in the Terminal, and DevTools shows real status codes (`200` , `304` , `404`) and headers such as `Content-Type` , the ones you studied in Chapter 2.
- **Test like it's published.** Addresses like `/` load `index.html` , missing files return a real `404`, and features that browsers block on `file://` pages (JavaScript modules and `fetch()`) work.
- **Catch mistakes before your visitors do.** A broken path shows up as a red `404` line in the log, and the log tells you exactly which address the browser asked for.

DOUBLE-CLICK THE FILE

LOCAL SERVER

GITHUB PAGES

	DOUBLE-CLICK THE FILE	LOCAL SERVER	GITHUB PAGES
The address bar shows	<code>file:///Users/you/web-projects/harbor-street-library/index.html</code>	<code>http://localhost:8000/</code>	<code>https://you.github.io/harbor-street-library/</code>
Uses HTTP (status codes, headers, the Network tab)	No: the browser just reads the file	Yes	Yes
An address ending in <code>/</code> loads <code>index.html</code>	No: you see a list of files	Yes	Yes
A path starting with <code>/</code>, such as <code>/css/styles.css</code>	Breaks: <code>/</code> means the top of your whole disk	Works: <code>/</code> means the folder the server started in	Breaks on project sites: <code>/</code> means <code>you.github.io</code> , not your project
JavaScript modules and <code>fetch()</code>	Blocked	Work	Work
Capital letters in file names	Ignored: a Mac treats <code>Logo.png</code> and <code>logo.png</code> as the same file	Ignored, for the same reason	Matter: the wrong case is a 404
Who can open it	Only you	Only you (or devices on your Wi-Fi, if you allow it)	Anyone on the internet



A rehearsal, not an exact copy

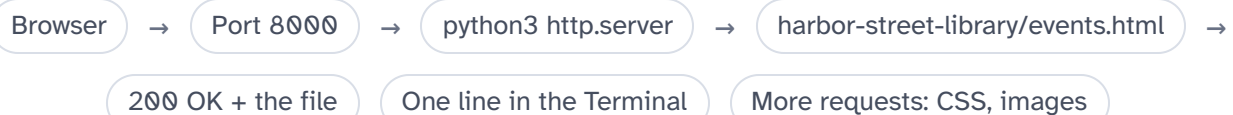
A local server behaves like GitHub Pages in the ways that matter most. Keep the last two rows in mind: a site can work locally and still break online if a file name's capital letters don't match, or if a path starts with `/`. Use relative paths such as `css/styles.css` and lowercase file names, and it will behave the same in both places.

What a local server does

A web server is a program that waits for requests on a **port**, a numbered door on your computer (Chapter 2 introduced these). When a request arrives, the server turns the path in the address into a file inside one folder, called its **document root**, and sends that file back. Python's built-in server uses whichever folder you're in when you start it as the document root. Step through one request:

WALKTHROUGH

One request to your local server



1

You type `http://localhost:8000/events.html`. `localhost` is the name for "this computer" (the loopback address `127.0.0.1`), so the request never leaves your Mac. No Wi-Fi or internet connection is needed.

2

The request arrives at port `8000`. Only one program can listen on a port at a time, and right now Python's server is that program.

3

The server takes the path `/events.html` and finds the file with that name inside its document root: the folder you were in when you started it. For `/` it looks for `index.html`.

4

It reads the file and sends it back with the status code `200 OK` and a `Content-Type` header (`text/html` for a `.html` file). If no file matches, it sends `404` instead.

5

It prints one line in the Terminal for the request: who asked, when, for what, and the status code it sent. This log is your window into the conversation.

6

The browser reads the HTML, finds the stylesheet and images it links to, and asks for each one. Each is a separate request with its own log line. One page usually means several requests.

i You've already used one

The Live Server extension in VS Code is a local server too. It answers at `http://127.0.0.1:5500/` and reloads the page every time you save. It's great while you type. The Terminal server on this page is plainer: it adds nothing to your pages and doesn't reload them for you, and it shows you every request. That makes it closer to a real hosting server, and better for seeing what's going on.

Start a server with Python, step by step

Every Mac can run **Python** once Apple's free Command Line Tools are installed (the same tools Git needs). Python includes a small web server called `http.server`, so there's nothing else to download. Do each step in order. Type each command exactly, then press Return.

1 Open Terminal

Press `Command` + `Space` to open Spotlight, type **Terminal** and press Return. (It also lives in **Applications → Utilities → Terminal**.) Terminal is a window where you type commands instead of clicking. The line ending in `%` is the **prompt**: it means Terminal is ready for your next command.

2 Check that Python is there

TYPE THIS

```
python3 --version
```

YOU SHOULD SEE SOMETHING LIKE

```
Python 3.9.6
```

python3

The Python 3 program. On a Mac it's always `python3`; plain `python` gives `zsh: command not found: python`.

--version

Asks it to print its version number and quit, a quick way to check that it's installed. Any 3.x version works.



A window asks to install "command line developer tools"

Choose **Install**, accept the license and wait a few minutes. When it finishes, run `python3 --version` again. These are the same tools Git uses, so if `git --version` already works on your Mac, you won't see this window.

3

Go to your site's folder

The server shares whichever folder Terminal is "in", so first move into the folder that holds your site's `index.html`. The example below uses a site folder called `harbor-street-library` inside a `web-projects` folder; use your own folder's path.

TYPE THESE ONE AT A TIME

```
cd ~/web-projects/harbor-street-library
ls
```

LS SHOULD LIST YOUR SITE'S FILES

```
css      events.html  images      index.html  report.html
```

cd ~/web-projects/harbor-street-library

change directory: move into that folder. `~` is short for your home folder (`/Users/yourname`). The prompt changes to show the folder's name.

ls

list the files in the current folder. Look for `index.html`. If it isn't listed, you're in the wrong folder.



Shortcut: drag the folder in

Type `cd` and a space, then drag your site's folder from Finder into the Terminal window. Terminal types the full path for you, with any spaces handled correctly. Press Return.

4

Start the server

TYPE THIS

```
python3 -m http.server 8000 --bind 127.0.0.1
```

TERMINAL ANSWERS

```
Serving HTTP on 127.0.0.1 port 8000 (http://127.0.0.1:8000/) ...
```

python3

Start Python.

-m http.server

`-m` means "run a module", a ready-made program that ships with Python. `http.server` is its built-in web server.

8000

The port to listen on. Any number from 1024 to 65535 that nothing else is using will do; 8000 and 8080 are the usual choices for development. Ports below 1024 are reserved, and Python answers `PermissionError: [Errno 13] Permission denied`.

--bind 127.0.0.1

Listen only on the loopback address, so only programs on this Mac can connect. Without it, other devices on the same Wi-Fi can open your site too (useful for [testing on your phone](#), but not on a coffee shop network).

Terminal looks stuck now, with no new prompt. That's normal: the server is running and waiting for requests. Leave this window open. To type other commands meanwhile, open a second Terminal tab with `Command + T`.

5 Open your site

In Chrome, Safari or Firefox, go to `http://localhost:8000/`. Your `index.html` appears, and new lines scroll into the Terminal, one per request:

TERMINAL, AFTER THE PAGE LOADS

```
127.0.0.1 - - [30/Sep/2026 14:02:11] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [30/Sep/2026 14:02:11] "GET /css/styles.css HTTP/1.1" 200 -
127.0.0.1 - - [30/Sep/2026 14:02:11] "GET /images/reading-room.jpg HTTP/1.1" 200 -
127.0.0.1 - - [30/Sep/2026 14:02:11] code 404, message File not found
127.0.0.1 - - [30/Sep/2026 14:02:11] "GET /favicon.ico HTTP/1.1" 404 -
```

Browsers ask for `/favicon.ico` (the little tab icon) on their own. If your site doesn't have one, that 404 is harmless. The next section explains every part of these lines. Open DevTools (`Command` + `Option` + `I`) and the **Network** tab, reload, and you'll see the same requests from the browser's side, with their status codes and headers.



Changed a file? Reload the page yourself

Unlike Live Server, this server doesn't refresh the page when you save. Save the file, then reload the browser (`Command` + `R`). If the old version sticks, force a fresh copy with `Command` + `Shift` + `R` .

6 Stop the server

Click in the Terminal window and press `Control` + `C` (the Control key, not Command). The server stops, and the prompt comes back:

TERMINAL

```
^C
Keyboard interrupt received, exiting.
```

Closing the Terminal window also stops it. Once it's stopped, `http://localhost:8000/` shows *"This site can't be reached. localhost refused to connect."*, because nothing is listening on that port anymore.

Read the server's log

Each log line is a short record of one HTTP request, the same request and response you studied in Chapter 2. Select any part of the line to see what it means.

EXPLORER

Anatomy of a log line

```
127.0.0.1 - - [30/Sep/2026 14:02:11] "GET /css/styles.css HTTP/1.1"
200 -
```

127.0.0.1

Who asked. The address of the computer that sent the request. `127.0.0.1` (or `:::1`) is this Mac. A phone on your Wi-Fi shows up as its own address, such as `::ffff:192.168.1.40`.

- -

Two unused fields. They are placeholders for a user name, left over from an older log format. They are always `-` here.

[30/Sep/2026 14:02:11]

When. The date and time the request arrived, in your Mac's local time.

"GET

The method. `GET` means "send me this". Links, images, stylesheets and ordinary page loads all use GET. A `HEAD` request asks for the headers only.

/css/styles.css

What was asked for. The path part of the URL. The server looks for this file inside its document root, so `/css/styles.css` means the `styles.css` file in the `css` folder of the folder you started the server in.

HTTP/1.1"

The protocol version the browser used: HTTP/1.1.

200

The status code the server sent back. `200` found it, `304` "use your saved copy", `404` not found. See the table below.

-

Response size. Python leaves this field as `-`.

Status codes you'll see

CODE	WHAT HAPPENED	WHAT TO DO
200	Found the file and sent it.	Nothing. This is success.
304	On a reload, the browser asked whether the file had changed since last time. It hadn't, so the server said "use your saved copy" and sent no body.	Nothing. It saves time. If you edited the file, the next reload gets a fresh <code>200</code> .
301	You asked for a folder without the final slash (<code>/css</code>). The server redirected the browser to <code>/css/</code> .	Nothing. The browser follows the redirect on its own.
404	No file matches that path. The line before it says <code>code 404, message File not found</code> .	Compare the path in the log with your real file names and folders. The log shows exactly what the browser asked for, typos included.
400	The request made no sense to the server, often because you typed <code>https://</code> for a server that only speaks plain <code>http://</code> .	Use <code>http://localhost:8000/</code> .

Practice: a Mac and a browser, side by side

Try everything here before (or while) you do it for real. The Terminal understands the commands from this page, and the browser beside it (below it on a phone) can only visit servers you start in that Terminal. The Terminal output matches what a real Mac prints. Work through the missions below the simulator; each one ticks off when you do it.

The interactive simulator is available in the online version of this page.

Optional: open your site on your phone

Want to see your site on a real phone before it's published? If the phone and the Mac are on the **same Wi-Fi network**, the phone can reach your Mac's server. Two things change: start the server without `--bind 127.0.0.1` so it accepts connections from other devices, and use your Mac's Wi-Fi address instead of `localhost`. On the phone, `localhost` means the phone itself.

IN A SECOND TERMINAL TAB (COMMAND + T): FIND YOUR MAC'S WI-FI ADDRESS

```
ipconfig getifaddr en0
```

IT PRINTS A PRIVATE ADDRESS SUCH AS

```
192.168.1.23
```

BACK IN YOUR SITE FOLDER: START THE SERVER FOR EVERY DEVICE ON THE NETWORK

```
python3 -m http.server 8000
```

TERMINAL ANSWERS (:: MEANS EVERY NETWORK ADDRESS ON THIS MAC)

```
Serving HTTP on :: port 8000 (http://[::]:8000/) ...
```

On the phone, open `http://192.168.1.23:8000/`, using the address your Mac printed. Requests from the phone show up in the log with the phone's address, such as `::ffff:192.168.1.40`. (The `::ffff:` part is how an IPv4 address is written inside IPv6. You can ignore it.) If macOS asks whether **python3** may accept incoming network connections, choose **Allow**.



Only on networks you trust

Without `--bind 127.0.0.1`, anyone on the same Wi-Fi can open every file in that folder, and in the folders inside it. Do this at home, never on public Wi-Fi, and press `Control` + `C` as soon as you're done. `192.168.x.x` addresses are private (Chapter 2), so nobody outside your network can reach them.

Troubleshooting

Each problem below starts with the exact message you'd see.

▼ `OSError: [Errno 48] Address already in use`

Another program is already listening on that port, often a server you started earlier in another Terminal window or tab and forgot. Two fixes:

1. The quick one: pick another port, for example `python3 -m http.server 8001 --bind 127.0.0.1`, and visit `http://localhost:8001/`.
2. The tidy one: find the other program and stop it. `lsof -nP -i :8000` lists what's using port 8000. The number in the **PID** column is its process ID, and `kill` followed by that number stops it. If the old server is in a Terminal window you can find, pressing `Control` + `C` there does the same thing.

EXAMPLE

```
lsof -nP -i :8000
```

COMMAND	PID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
Python	50123	student	5u	IPv6	0x4c1f0b2e9d7a3c11		0t0	TCP *:8000 (LISTEN)

```
kill 50123
```

▼ The browser shows "Directory listing for /"

The server started in a folder that has no `index.html`, so it lists the folder's contents instead. Usually you're one folder too high (in `web-projects` instead of `harbor-street-library`). You can click through the list to your site, but it's better to stop the server, `cd` into the right folder and start it again. Also check the file's name: it must be exactly `index.html`, not `Index.html`, `index.htm` or `index.html.txt`. GitHub Pages never shows a listing; it answers 404.

▼ "This site can't be reached. localhost refused to connect."

Nothing is listening on that port. Either the server isn't running (you pressed Control + C, or closed the Terminal window), or the port in the address doesn't match the one you started. Look in Terminal: a running server shows its `Serving HTTP on ...` line with no prompt after it.

▼ "This site can't provide a secure connection" (ERR_SSL_PROTOCOL_ERROR)

The address starts with `https://`, but this server only speaks plain `http://`. The log shows a `code 400` line full of strange characters, which is the encrypted greeting the server couldn't read. Type `http://localhost:8000/`.

▼ A page loads, but a stylesheet or image is missing (404 in the log)

Read the path in the 404 line: it's exactly what the browser asked for. Compare it letter by letter with the real file and folder names, and with the `href` or `src` in your HTML. Common causes are a wrong folder (`image/` vs `images/`), a missing `../`, or a typo in the file extension.

▼ It works on my local server but breaks on GitHub Pages

Two usual suspects. **Capital letters:** a Mac ignores case, so `Images/Logo.PNG` finds `images/logo.png` locally, but GitHub Pages treats them as different files. **A leading slash:** `/css/styles.css` means the top of `you.github.io` online, not your project folder. Use lowercase names and relative paths.

▼ `zsh: command not found: python`

Type `python3`, with the 3. Macs don't include a plain `python` command.

▼ Terminal "does nothing" after I start the server

That's the server running. It keeps the window busy until you stop it, and the prompt only returns after Control + C. For other commands, open a new tab with `Command` + `T`.

Other servers on your Mac

Python's server is the simplest way to see a real server at work, but it isn't the only one.

Live Server (VS Code)

What it is. A VS Code extension that runs a local server on `http://127.0.0.1:5500/` and reloads the page every time you save.

When to use it. While you're writing HTML and CSS and want to see every change immediately. Right-click an HTML file and choose **Open with Live Server**, or click **Go Live** in the status bar.

Watch out. It adds a small script to each page so it can reload it (you'll see it in DevTools, but not in your file), and it serves the folder you opened in VS Code, so open your site folder itself. It shows no request log.

Python http.server

What it is. The server this page teaches, built into Python.

When to use it. To test the finished site the way a host would serve it, to watch the requests in the log, to check a site without VS Code, or to open it on your phone.

Watch out. It's meant for development only: it's simple, has no security features and shows directory listings. Never use it to host a real public website.

Apache (built into macOS)

What it is. Apache HTTP Server, one of the most widely used production web servers (the "A" in the LAMP stack from Chapter 2), comes preinstalled on macOS. Its document root is `/Library/WebServer/Documents`.

When to use it. Out of curiosity, to see the same server software many real websites run. For coursework, Python's server is simpler.

START IT (ASKS FOR YOUR MAC'S PASSWORD; NOTHING APPEARS AS YOU TYPE IT)

```
sudo apachectl start
```

Then visit `http://localhost/` (port 80, so no number is needed) and you'll see **It works!**. When you're done:

STOP IT

```
sudo apachectl stop
```

Watch out. `sudo` runs the command as the Mac's administrator, and Apache keeps running after you close Terminal until you stop it. Putting your own files into its document root also needs administrator rights. Stop it when you're done.

Node.js (later)

What it is. If you install Node.js later in your studies, `npm serve` starts a similar development server with one command. Many front-end tools you'll meet in later courses include their own development servers too.

When to use it. Once you're writing JavaScript projects that already use Node. It isn't needed for plain HTML and CSS sites.

Cheat sheet

COMMAND OR KEY	WHAT IT DOES
<code>cd ~/web-projects/your-site</code>	Move into your site's folder (the one with <code>index.html</code>)
<code>ls</code>	List the files in the current folder
<code>pwd</code>	Print the full path of the folder you're in
<code>python3 -m http.server 8000 --bind 127.0.0.1</code>	Serve the current folder at <code>http://localhost:8000/</code> , for this Mac only
<code>python3 -m http.server 8000</code>	Serve it to every device on your Wi-Fi as well (trusted networks only)
<code>python3 -m http.server 8000 --directory ~/web-projects/your-site</code>	Serve a folder without <code>cd</code> -ing into it first
<code>Control</code> + <code>C</code>	Stop the running server
<code>Command</code> + <code>T</code>	Open another Terminal tab while the server runs
<code>lsof -nP -i :8000</code>	Show which program is using port 8000, and its PID
<code>kill 50123</code>	Stop the program with that PID
<code>ipconfig getifaddr en0</code>	Print your Mac's Wi-Fi address, for phone testing